DEVOPS BOOTCAMP

Containerization · Module

Docker & Containers

Build once. Ship anywhere. Run everywhere.

Sagyam Thapa

PART 1

Foundations

What a container really is, why it beats a VM for our use cases, and how the runtime stack fits together.

THE CORE PROBLEM

Why do we need containers?

THE PAIN

- "Works on my machine" — runs in dev, breaks in prod
- Dependency hell — app A needs Python 3.9, app B needs 3.12, on one host
- Environment drift — snowflake servers nobody can reproduce
- Slow onboarding — days to configure a new dev box

THE IDEA

- Package the app with its entire environment — runtime, libraries, config
- Bundle it into one immutable, self-contained unit
- That unit runs identically on any machine with a container runtime

That portable, self-contained unit is a container.

DEFINITION

What exactly is a container?

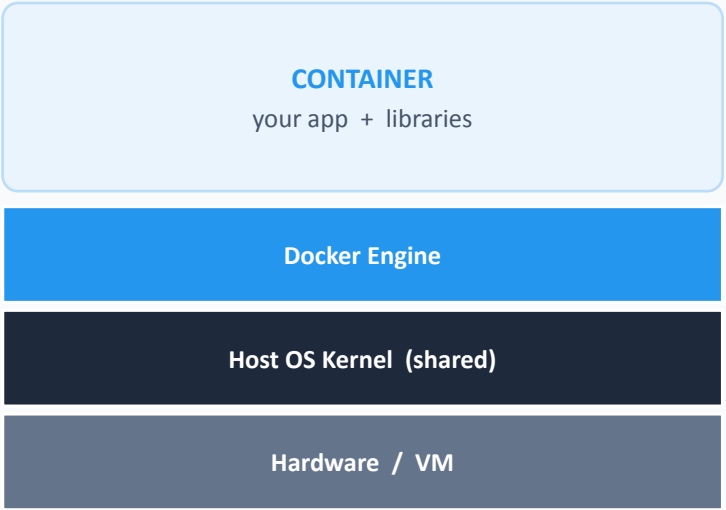
Isolated process — its own filesystem, network & process tree

Namespaces — the kernel feature that provides that isolation

cgroups — limit CPU, memory & I/O per container

Shares the host kernel — no guest OS, unlike a VM

Built from an image — a read-only template

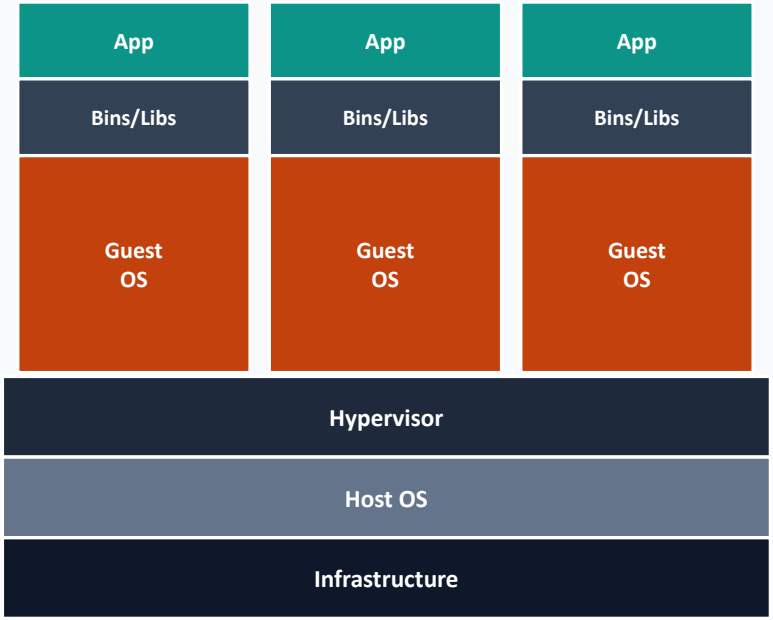


Docker is the platform that builds, ships & runs containers — the container is the unit.

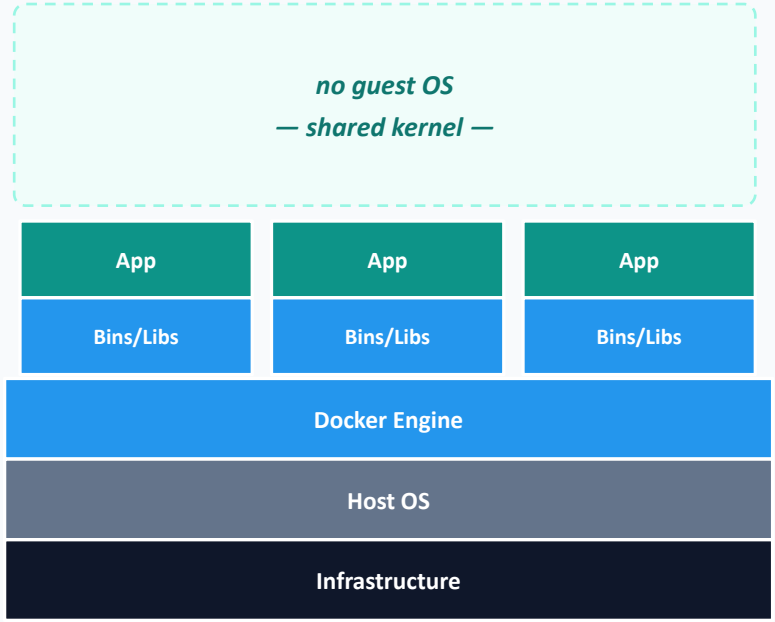
ARCHITECTURE

Containers vs Virtual Machines

VIRTUAL MACHINES



CONTAINERS



DECISION GUIDE

VM vs Container — the trade-offs

Dimension	Virtual Machine	Container
Isolation	Strong — full hardware virtualization	Process-level — shared kernel
Footprint	Gigabytes (full guest OS)	Megabytes (app + libs)
Startup	Seconds to minutes	Milliseconds to seconds
Density	Tens per host	Hundreds per host
Performance	Hypervisor overhead	Near-native
Portability	Hypervisor-specific images	Any OCI runtime
Best for	Strong isolation, mixed OSes	Microservices, scale, CI/CD

Complementary, not rivals — in the cloud, containers usually run inside VMs.

WHY TEAMS ADOPT IT

The payoff of containers

Consistency
Same image runs identically on laptop, CI and prod

Fast startup
Boot in milliseconds, not minutes like a VM

Lightweight
Share the kernel — MBs, not GBs, per app

High density
Pack far more workloads onto one host

Portable
Runs on any host with an OCI runtime

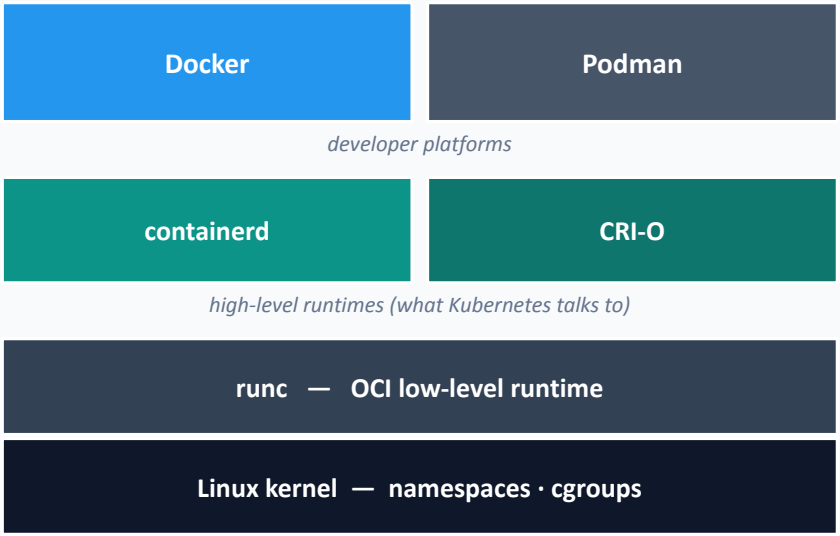
Reproducible
Immutable images mean no config drift

Easy scaling
Spin identical replicas up and down on demand

Safe rollback
Redeploy the previous image tag to revert

THE ECOSYSTEM

Docker is one of several runtimes



OCI — open standard for image & runtime formats

runc — actually spawns the container process

containerd / CRI-O — manage images & lifecycle

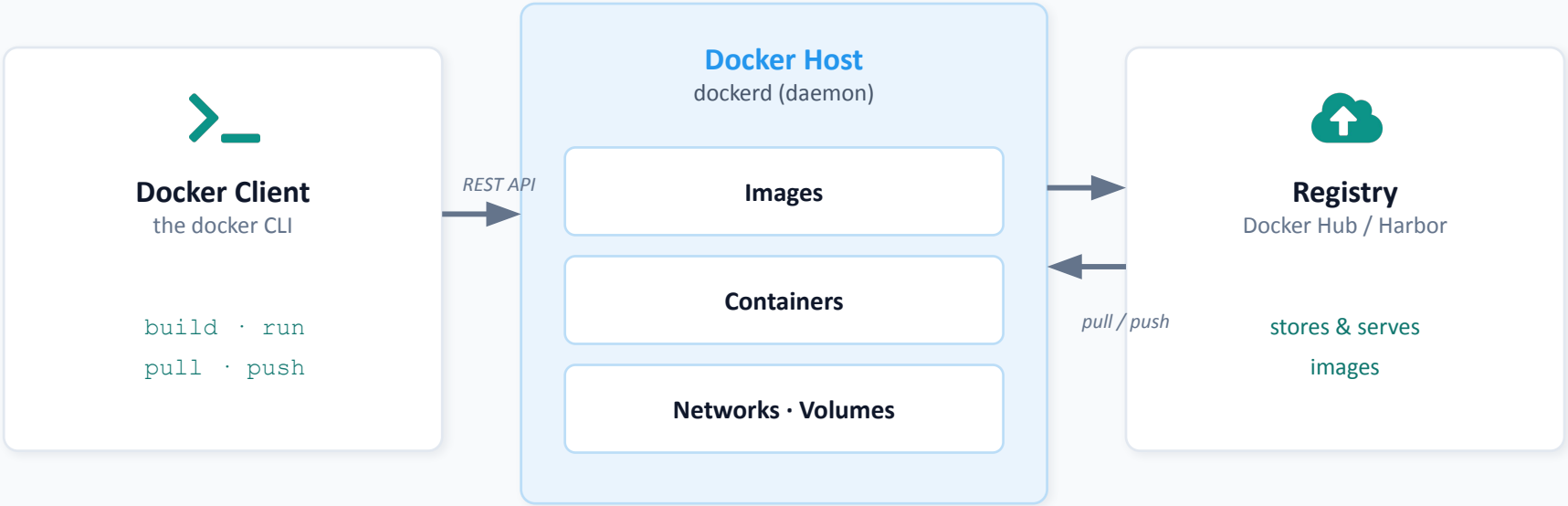
Docker / Podman — full platforms on top (Podman is daemonless)

Kubernetes dropped Docker as its runtime — it uses containerd directly. Your images still run fine.

Docker · Podman · containerd · CRI-O · runc · gVisor · Kata

HOW IT FITS TOGETHER

The Docker architecture



The CLI just sends commands; the daemon does the real work over a socket / REST API.

PART 2

Install & Operate

Getting Docker running, the image-to-container model, and the everyday commands you will use in every session.

GETTING STARTED

Installing & configuring Docker

install.sh

```
# Ubuntu — official apt repository
$ sudo apt-get update
$ sudo apt-get install ca-certificates curl
# add Docker's GPG key + repo (see docs)
$ sudo apt-get install docker-ce docker-ce-cli \
    containerd.io docker-compose-plugin

# use docker without sudo
$ sudo usermod -aG docker $USER
# log out / in, then verify
$ docker run hello-world
Hello from Docker!
```

Engine, not Desktop — on Linux servers install Docker Engine (CE)

docker group = root — members can take over the host; treat as sudo

Rootless mode — run the daemon unprivileged on shared boxes

daemon.json — configure logging, registries, storage

Start on boot — systemctl enable --now docker

THE KEY MENTAL MODEL

Image → Container

Image — a read-only template: app + libs + filesystem, in layers

Container — a running instance of an image + a thin writable layer

One image → many containers — start as many identical copies as you need

Immutable — you don't edit an image, you rebuild it

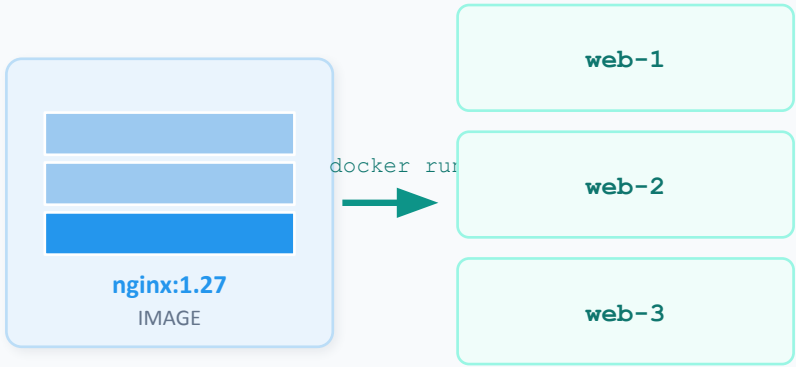


Image is the class; container is the object. Image is the recipe; container is the dish.

WORKING WITH IMAGES

Search, pull, list & inspect

images

```
$ docker search nginx
NAME     STARS     OFFICIAL
nginx    20000     [OK]

$ docker pull nginx:1.27  # pin a tag
Downloaded newer image for nginx:1.27

$ docker images
REPOSITORY  TAG     SIZE
nginx       1.27    188MB

$ docker rmi nginx:1.27  # remove image
```

Image name registry/repository:tag (default docker.io/library, :latest)

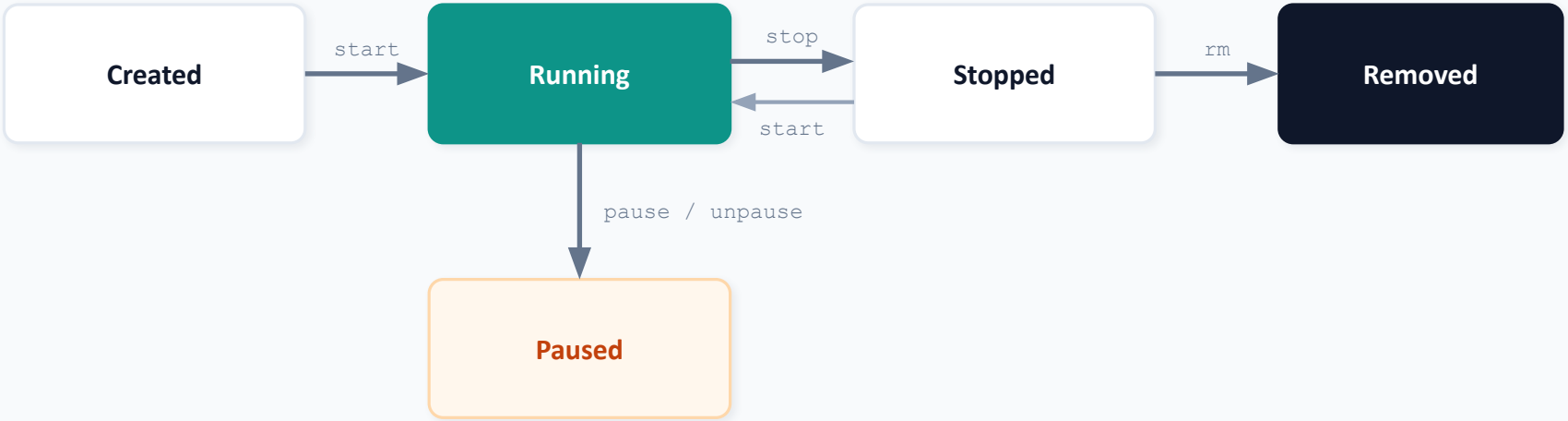
Avoid :latest it moves — pin a version for reproducible builds

Digests pull by @sha256:… to lock the exact bytes

Layers are shared a second image reuses common layers on disk

THE CONTAINER LIFECYCLE

States & the commands between them




```
$ docker run -d --name web nginx   # create + start in one step (you'll use this 90% of the time)
```

THE WORKHORSE COMMAND

Running containers in practice

```
docker run

$ docker run -d --name web \
  -p 8080:80 \
  -e NODE_ENV=prod \
  -v site:/usr/share/nginx/html \
  nginx:1.27
a1b2c3d4...   # container id

$ docker run -it --rm ubuntu bash  # throwaway
```

-d detached — run in the background

--name a stable name to reference it

-p H:C publish host port → container port

-e set an environment variable

-v mount a volume for persistent data

-it / --rm interactive shell / auto-remove on exit

docker run pulls the image if missing, creates a container, then starts it.

EXPOSING AN APPLICATION

Publishing a container's port

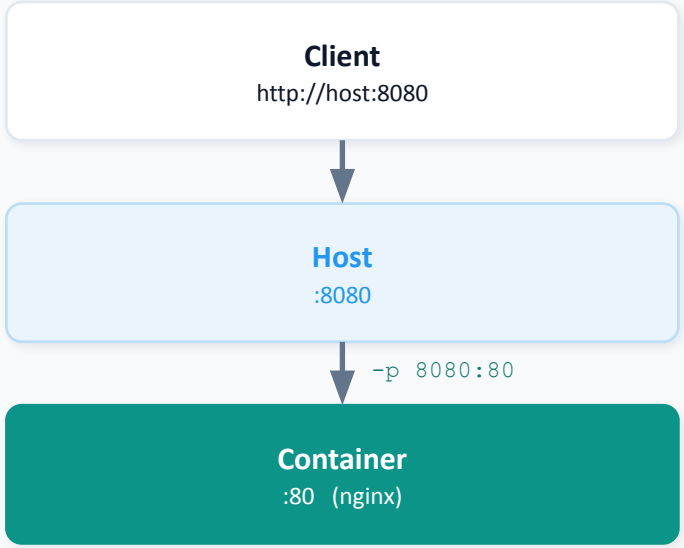
-p 8080:80 maps host port 8080 → container port 80

EXPOSE in a Dockerfile only documents a port — it does NOT publish

-P publishes all exposed ports to random host ports

127.0.0.1:8080:80 bind to localhost to keep a port private

docker port web lists the active mappings



$ docker run -d -p 8080:80 nginx → curl localhost:8080 returns the nginx welcome page

OBSERVE & TIDY UP

Inspecting and cleaning up

inspect & clean

```
$ docker ps
$ docker ps -a  # include stopped
$ docker logs -f web
$ docker exec -it web bash
$ docker stats

# cleanup
$ docker stop web && docker rm web
$ docker rmi nginx:1.27
$ docker system prune -a  # reclaim all unused
Total reclaimed space: 2.4GB
```

ps -a stopped containers still take disk space

logs -f stream stdout/stderr live

exec -it open a shell inside a running container

Dangling images untagged <none> layers — prune them

prune is destructive -a removes unused images too; be deliberate

PART 3

Build Images with Dockerfile

Turning a hand-run container into a reproducible recipe — instructions, layers, and the practices that keep images small and safe.

THE PROBLEM

Hand-built containers don't scale

THE FRAGILE WAY

- docker commit snapshots a container you set up by hand
- No record of what you did — undocumented & unrepeatable
- Not in version control: not reviewable, not diffable
- The next person (or future you) can't reproduce it

THE DOCKERFILE WAY

- A plain-text recipe that builds the image from scratch
- Lives in git next to your code — reviewed, versioned, diffable
- docker build produces the identical image on any machine

If it isn't in the Dockerfile, it doesn't exist.

ANATOMY

A Dockerfile, line by line

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

FROM the base image to build on

WORKDIR sets the working directory

COPY deps first manifests before source → better caching

RUN execute build steps (install deps)

ENV set runtime environment variables

EXPOSE document the listening port

USER drop to a non-root user

CMD default command when the container starts

REFERENCE

Dockerfile instructions at a glance

Instr.	Purpose
FROM	base image to start from
RUN	execute a command at build time
COPY	copy files from build context
ADD	COPY + URL / tar auto-extract
WORKDIR	set working directory
ENV	set environment variable
ARG	build-time variable

Instr.	Purpose
CMD	default command / args
ENTRYPOINT	the executable to run
EXPOSE	document a port
VOLUME	declare a mount point
USER	set the runtime user
LABEL	add metadata
HEALTHCHECK	container health probe

Build-time: FROM · RUN · COPY · ARG ***Run-time: CMD · ENTRYPOINT · ENV · USER***

THE CLASSIC CONFUSION

CMD vs ENTRYPOINT

CMD

- Provides default arguments
- Easily overridden: docker run img <other>
- Often the whole command for simple images

ENTRYPOINT

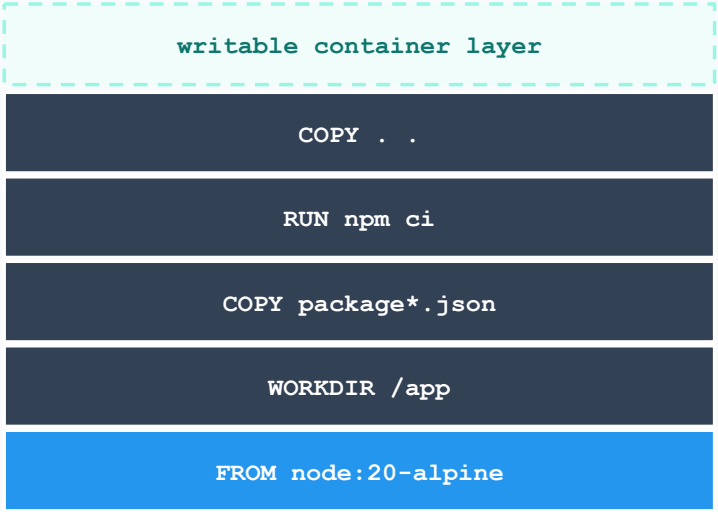
- Defines the executable that always runs
- CMD / CLI args are appended to it
- Override only with --entrypoint

best of both — exec form preferred

```
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8000"]
# docker run img              ->  python app.py --port 8000
# docker run img --port 9000  ->  python app.py --port 9000
```

PERFORMANCE

Layers & the build cache



read-only image layers

Every instruction adds a layer content-addressed & stacked

Layers are cached unchanged layers are reused on rebuild — fast

Order matters put rarely-changing steps first

Cache busts cascade change one layer → everything after it rebuilds

The pattern COPY manifests → install deps → COPY source last

Writable layer the thin top layer; lost when the container is removed

KEEP IMAGES SMALL & FAST

Dockerfile best practices

multi-stage build

```
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN go build -o app

# tiny final image — just the binary
FROM gcr.io/distroless/static
COPY --from=build /src/app /app
USER nonroot
ENTRYPOINT ["/app"]
```

Multi-stage builds compile in a fat image, ship only the artifact

Small base images alpine, slim, or distroless

.dockerignore keep junk & secrets out of the context

Order for cache dependencies before source

Combine RUN steps fewer layers; clean apt lists in the same layer

Pin versions reproducible builds; avoid :latest

One concern per image a container does one job

SECURITY BEST PRACTICES

Hardening images & containers

Run as non-root

USER directive; never run as root inside

Minimal base image

fewer packages = smaller attack surface

Scan every image

Trivy / Grype / Docker Scout in CI

No secrets in layers

use build secrets & runtime env, not COPY

Drop capabilities

--cap-drop ALL, read-only root filesystem

Patch & update

rebuild on base-image CVE fixes; sign images

PART 4

Networking & Storage

How containers talk to each other and the outside world, and how to keep data alive when containers do not.

CONTAINER NETWORKING

How containers get on the network

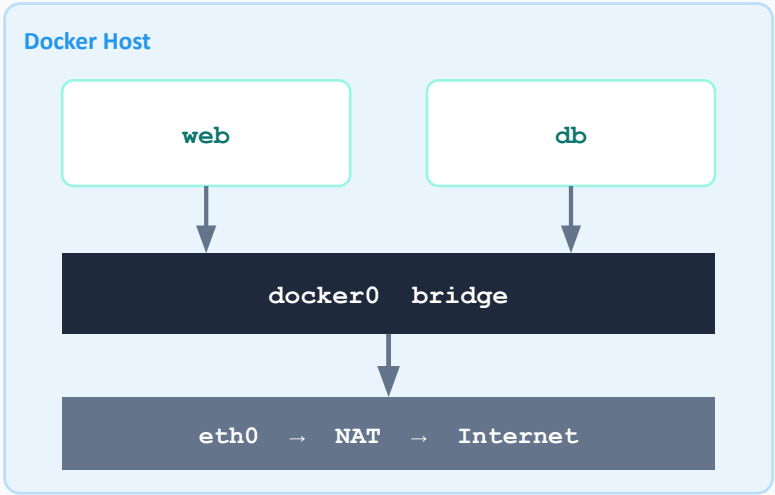
Own network namespace each container gets its own IP & interfaces

Virtual bridge Docker creates docker0; containers attach via veth pairs

Built-in DNS on a user-defined network, reach others by name

NAT to outside outbound traffic is masqueraded through the host

Ports stay private until you publish them with -p



Each container is networked in isolation; the bridge connects them and NAT reaches the world.

NETWORK DRIVERS

Types of container networking

Driver	Scope	When to use
bridge	single host	default; isolated apps on one host
host	single host	share host's stack; max performance, no isolation
none	single host	no networking; fully isolated
overlay	multi-host	connect containers across a Swarm cluster
macvlan	single host	give a container its own MAC/IP on the LAN

Prefer a user-defined bridge over the default — you get automatic DNS by container name.

NETWORKING IN PRACTICE

Connecting containers by name

network

```
$ docker network create appnet
$ docker network ls
NAME      DRIVER    SCOPE
appnet    bridge    local

$ docker run -d --name db --network appnet postgres
$ docker run -d --name api --network appnet myapi

# api reaches the database simply as:
  postgres://db:5432
```

User-defined bridge create one per app stack

DNS by name api connects to 'db', never a hard-coded IP

Isolation only containers on appnet can talk to db

Compose does this one network per project, automatically

THE PROBLEM

Containers forget everything

EPHEMERAL BY DESIGN

- The writable layer lives and dies with the container
- docker rm the container ⟶ all its data is gone
- Copy-on-write is slow for heavy writes (databases)
- Data is trapped inside one container — not shareable

PERSIST OUTSIDE IT

- Mount storage that outlives any single container
- Share the same data across containers and restarts
- Back it up and manage it independently of the image

Code belongs in the image; data belongs in a volume.

PERSISTENT STORAGE

Volumes, bind mounts & tmpfs

Volume

- Docker-managed storage
- Best for persistent app data
- Portable & backup-friendly

Bind mount

- Maps a host path inside
- Great for live code in dev
- Host-path dependent

tmpfs

- In-memory, never on disk
- For secrets & scratch data
- Vanishes when stopped

mounting

```
$ docker volume create pgdata
$ docker run -d -v pgdata:/var/lib/postgresql/data postgres  # volume
$ docker run -v $(pwd):/app node  # bind mount for dev
$ docker run --tmpfs /tmp:rw,size=64m alpine  # tmpfs
```

PART 5

Registries

Where images live and travel — Docker Hub, your own private registry, and scanning images before they ship.

WHERE IMAGES LIVE

Docker registry & Docker Hub

Registry stores & distributes images (push / pull)

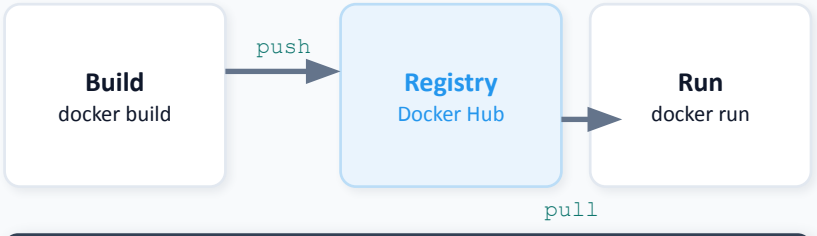
Docker Hub the default public registry (docker.io)

Repositories & tags user/app:1.0, user/app:latest

Official & verified curated base images you can trust

Public vs private keep proprietary images private

Hub rate limits anonymous pulls are throttled — log in or mirror



publish

```
$ docker login
$ docker tag myapi:1.0 user/myapi:1.0
$ docker push user/myapi:1.0
1.0: digest: sha256:... pushed
```

PRIVATE & SECURE

Self-hosted registry, Harbor & scanning

REGISTRY:2 (DISTRIBUTION)

- Minimal — just stores & serves images
- No web UI, RBAC, or scanning
- Runs in a single container
- Good for a simple private mirror

HARBOR (ENTERPRISE)

- Web UI + RBAC, projects & quotas
- Replication across registries
- Built-in Trivy vulnerability scanning
- Image signing (Cosign) + pull policies

Scanner gate: on push, Trivy scans every layer → policy blocks pulling images with critical CVEs.

PART 6

Compose, CI/CD & Orchestration

From one container to a whole stack, into a pipeline, and out across a cluster.

THE PROBLEM

Real apps are many containers

BY HAND IT'S PAINFUL

- A real app = web + API + database + cache + queue
- Long docker run commands, right order, right network
- Easy to forget a flag; impossible to hand off cleanly
- No single source of truth for the whole stack

DECLARE THE WHOLE STACK

- One YAML file describes every service, network & volume
- docker compose up brings the entire app up together
- Versioned in git — the stack is reproducible

One file. One command. The whole application.

DECLARATIVE STACKS

A compose file, explained

compose.yaml

```yaml
services:
  web:
    build: .
    ports: ["8080:80"]
    depends_on: [db]
  db:
    image: postgres:16
    environment: [POSTGRES_PASSWORD=secret]
    volumes: [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

services each container in the stack

build / image from a Dockerfile or a pulled image

depends_on start order between services

networks auto-created; talk by service name

volumes named, persistent storage

run it

```
$ docker compose up -d
$ docker compose ps
$ docker compose down
```

PROVISIONING SERVICES

Compose beyond the basics

.env files keep config & secrets out of the compose file

healthcheck + depends_on wait until db is actually ready, not just started

restart: unless-stopped auto-recover crashed services

Scaling run N identical replicas of a service

Override files compose.override.yaml for dev vs prod

Profiles optionally enable extra services (debug tools)

operate the stack

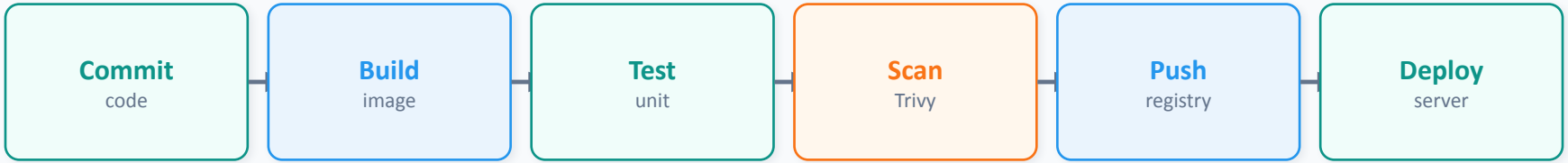
```
$ docker compose up -d --scale web=3
Creating web-1 ... web-2 ... web-3

$ docker compose logs -f web
$ docker compose exec db psql -U postgres

$ docker compose -f compose.yaml \
   -f compose.prod.yaml up -d
stack is up: web x3, db x1
```

INTO THE DEVOPS LOOP

Deploy via a CI/CD pipeline



.github/workflows/deploy.yml (excerpt)

```
$ docker build -t $REG/app:$SHA .
$ trivy image --exit-code 1 $REG/app:$SHA # fail on critical CVEs
$ docker push $REG/app:$SHA
$ ssh prod 'docker compose pull && \
          docker compose up -d'
deployed app:$SHA to prod
```

THE PROBLEM

One host is not enough

A SINGLE DOCKER HOST

- One machine = a single point of failure
- Limited CPU / memory — can't scale past one box
- Crashed container? Nothing restarts it for you
- Scaling & updates are manual and risky

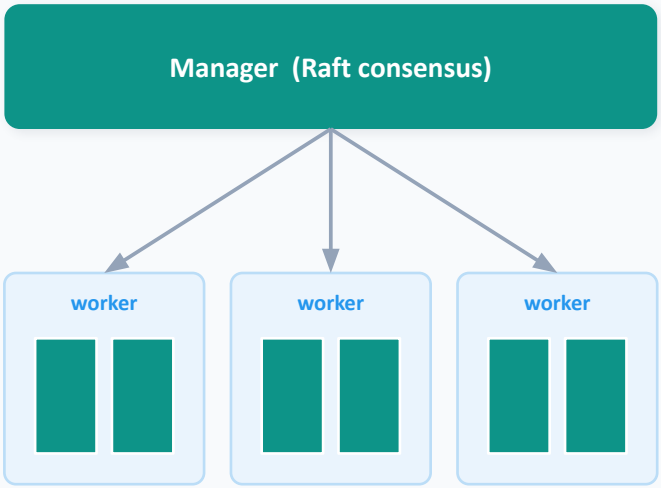
ORCHESTRATION

- Run containers across a cluster of machines
- Scheduling, load-balancing & self-healing built in
- Declare desired state; the orchestrator maintains it
- Rolling updates & rollbacks with little downtime

Tell it what you want running; it keeps it running.

BUILT-IN ORCHESTRATION

Docker Swarm



desired state → scheduled tasks (replicas)

Managers & workers managers hold state via Raft; workers run tasks

Services & replicas declare 'run 3 copies'; Swarm schedules them

Self-healing a dead task is rescheduled automatically

Routing mesh hit any node:port → routed to a healthy replica

Rolling updates update gradually; roll back on failure

swarm

```
$ docker swarm init
$ docker service create --replicas 3 -p 80:80 nginx
overall progress: 3 out of 3 tasks
```

THE LANDSCAPE

Swarm vs Kubernetes

DOCKER SWARM

- Built into Docker — nothing extra to install
- Simple: few concepts, fast to learn
- Great for small / medium clusters
- Smaller ecosystem & community

KUBERNETES

- The industry standard at scale
- Very powerful & extensible (operators, CRDs)
- Huge ecosystem & cloud support
- Steep learning curve — runs your same images

Same containers underneath — Kubernetes is the next chapter.

PUTTING IT TOGETHER

The Docker workflow

BUILD

- Dockerfile
- docker build
- layers & cache

SHIP

- tag & push
- scan in CI/CD
- to a registry

RUN

- docker run
- compose up
- swarm / k8s

Write a recipe, build an image, ship it through a registry & pipeline, run it anywhere — from one container to a cluster.

CHAPTER COMPLETE

From containers to clusters

You can now package any app, ship it through a registry and a pipeline, and run it anywhere — reproducibly.

docker build | run -p | volume | compose up | push | service create

Up next: **Kubernetes — orchestration at scale**